

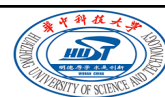
人工智能与自动化学院

模式识别与机器学习

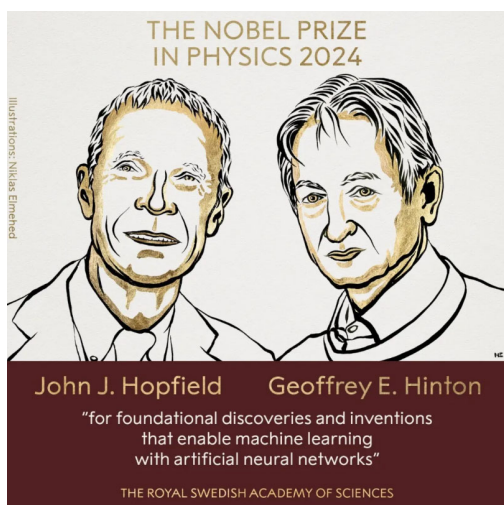
授课教师：杨卫东 胡静



AI时代



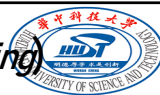
2024 年诺贝尔物理学奖、诺贝尔化学奖



人工智能与自动化学院

2

第十讲 神经网络到深度学习(*From Neural Networks to Deep Learning*)



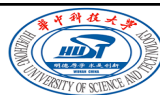
10.1 神经网络动机 (*Motivation of Neural Networks*)

10.2 神经网络模型 (*Neural Network Hypothesis*)

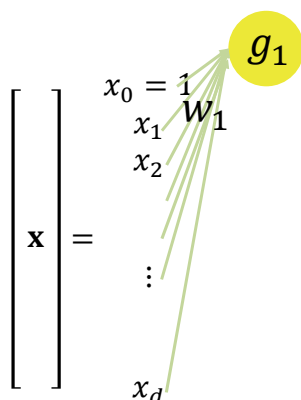
10.3 神经网络学习 (*Neural Network Learning*)

10.4 深度神经网络 (*Deep Neural Networks*)

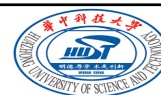
10.1 神经网络动机



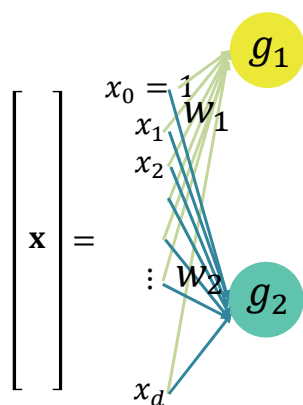
多个感知器的线性集成(*Linear Aggregation of Perceptrons*)



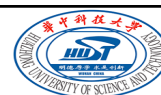
10.1 神经网络动机



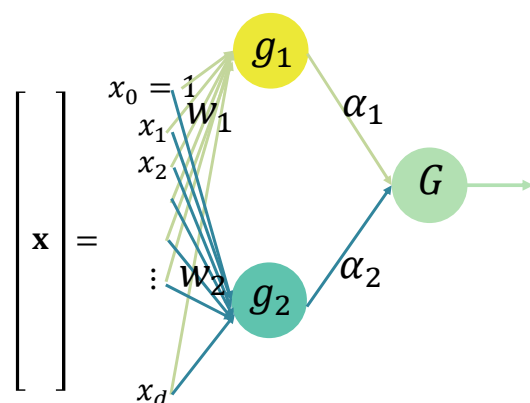
多个感知器的线性集成(Linear Aggregation of Perceptrons)



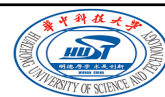
10.1 神经网络动机



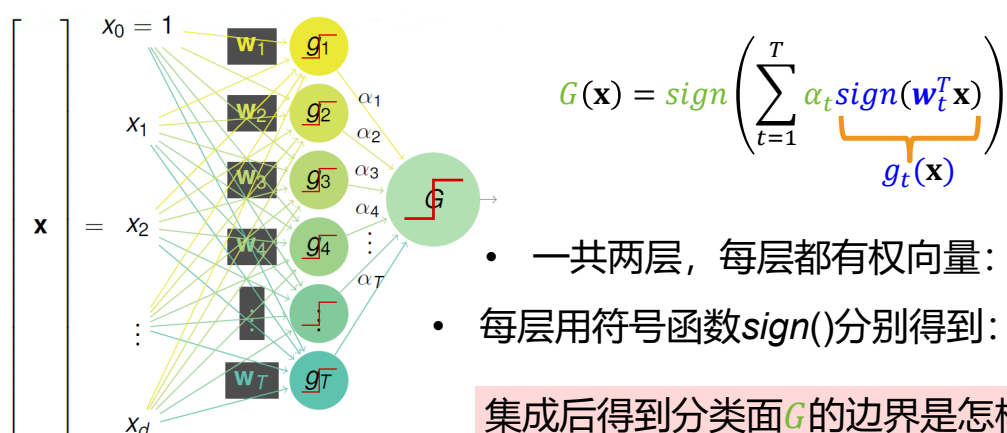
多个感知器的线性集成(Linear Aggregation of Perceptrons)



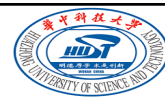
10.1 神经网络动机



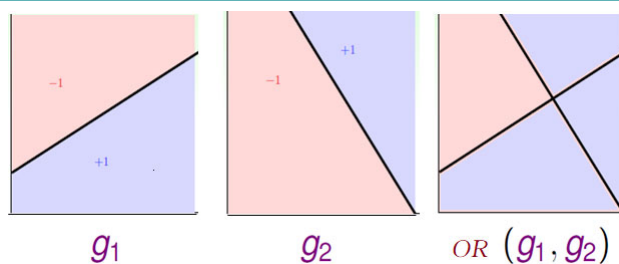
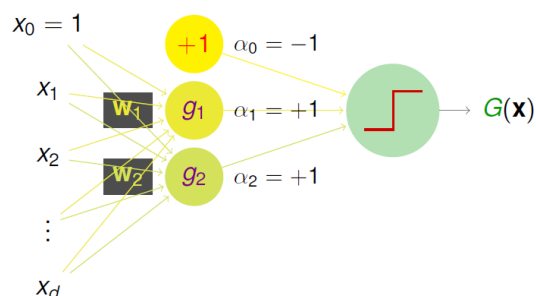
多个感知器的线性集成(Linear Aggregation of Perceptrons)



10.1 神经网络动机



用集成方式实现逻辑运算



$$G(\mathbf{x}) = \text{sign}(-1 + g_1(\mathbf{x}) + g_2(\mathbf{x}))$$

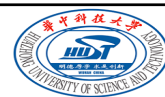
$$\text{if } g_1(\mathbf{x}) = g_2(\mathbf{x}) = +1: \quad G(\mathbf{x}) = +1$$

$$\text{otherwise:} \quad G(\mathbf{x}) = -1$$

集成也能实现逻辑运算 **OR**、**NOT**

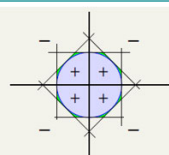
$$G(\mathbf{x}) \equiv AND(g_1(\mathbf{x}), g_2(\mathbf{x}))$$

10.1 神经网络动机

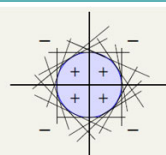


两层集成的强大能力:

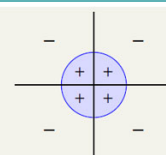
足够多的感知器可得到光滑的非线性分类面



8 perceptrons



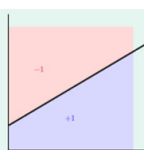
16 perceptrons



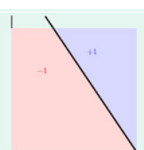
target boundary

两层集成的不足:

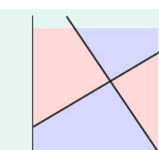
在两层结构下, 做不到“线性可分”, 无法实现“异或(XOR)”逻辑



g_1

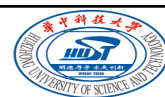


g_2



$XOR(g_1, g_2)$

10.1 神经网络动机

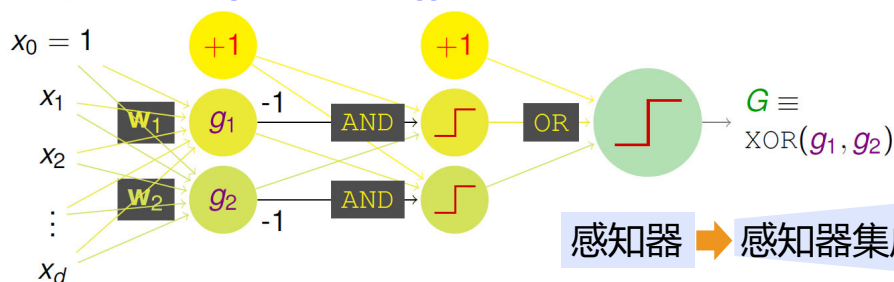


如何实现“异或”逻辑?

- 途径1: 非线性变换
- 途径2: 在两层实现简单逻辑基础上, 再增加层数

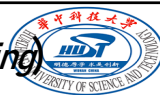
$$XOR(g_1, g_2) = OR(AND(-g_1, g_2) + AND(g_1, -g_2))$$

多层感知机—基本的神经网络



感知器 → 感知器集成 → 多层感知机

第十讲 神经网络到深度学习 (From Neural Networks to Deep Learning)



10.1 神经网络动机 (Motivation of Neural Networks)

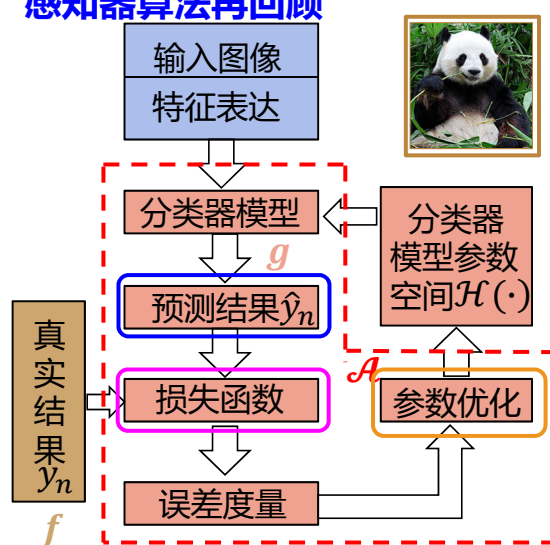
10.2 神经网络模型参数空间 (Neural Network Hypothesis)

10.3 神经网络学习 (Neural Network Learning)

10.4 深度神经网络 (Deep Neural Networks)

10.2 神经网络模型参数空间

感知器算法再回顾



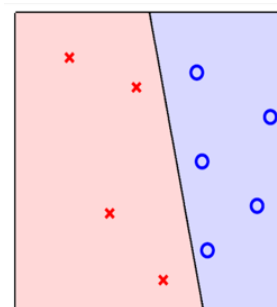
$$\hat{y}_{n(t)} = \text{sign}(\mathbf{w}_t^T \mathbf{x}_{n(t)})$$

算法收敛:

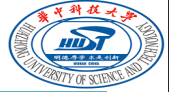
$$L_{in} = \sum_{n=1}^N \mathbb{I}[y_n \neq \hat{y}_n] = 0$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + y_n \mathbf{x}_{n(t)}$$

线性可分

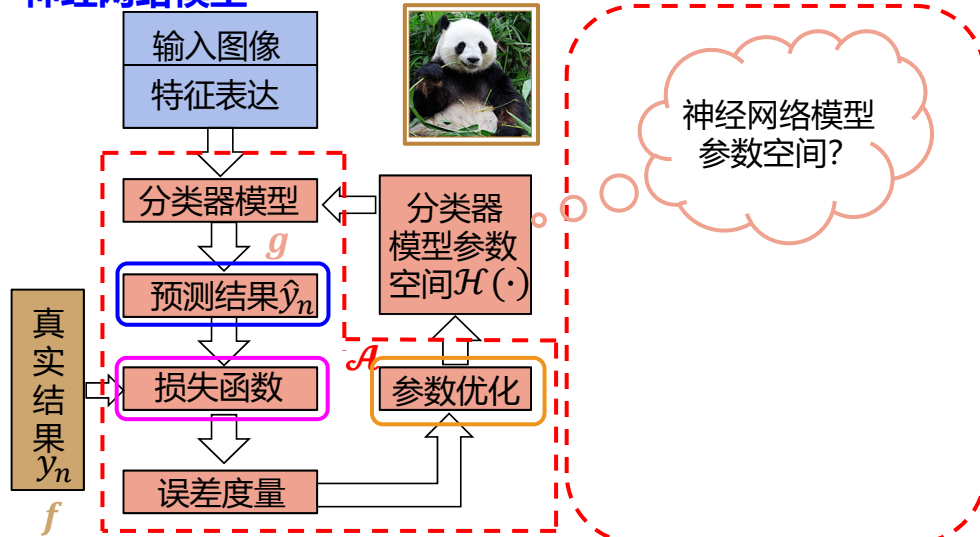


- 设置初始分类面 (权重) $\mathbf{w}_0$
- 如果有样本分错, 就修正权重



10.2 神经网络模型参数空间

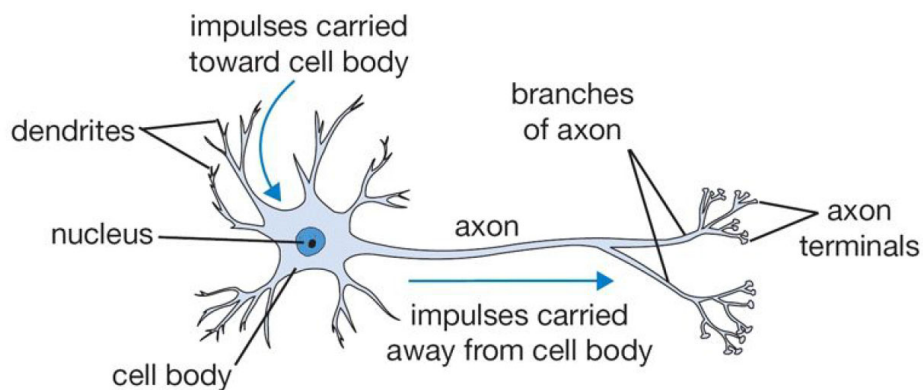
神经网络模型



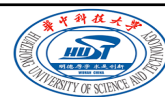
10.2 神经网络模型参数空间



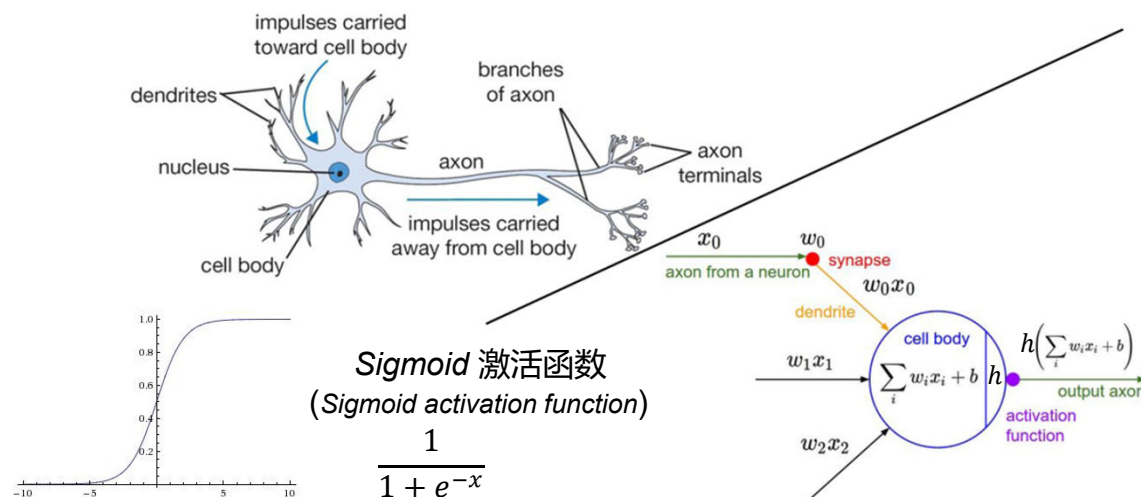
神经网络(Neural Networks):



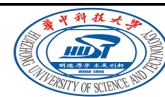
10.2 神经网络模型参数空间



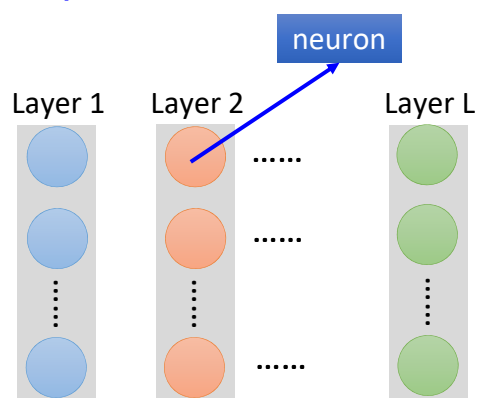
神经网络(Neural Networks):



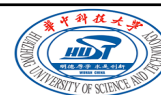
10.2 神经网络模型参数空间



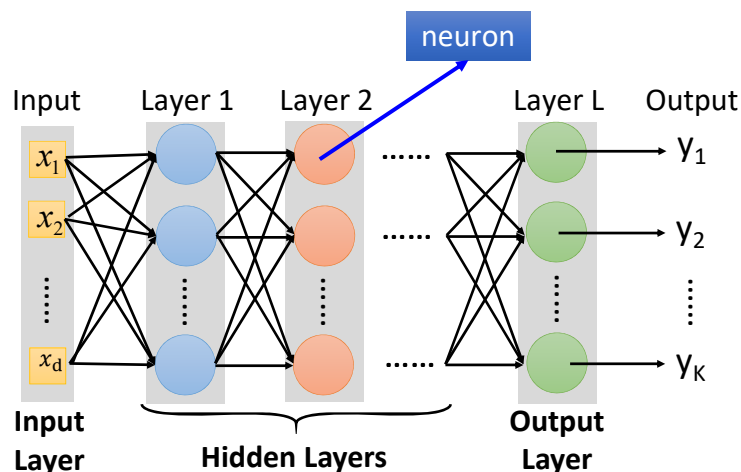
神经网络(Neural Networks):



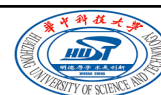
10.2 神经网络模型参数空间



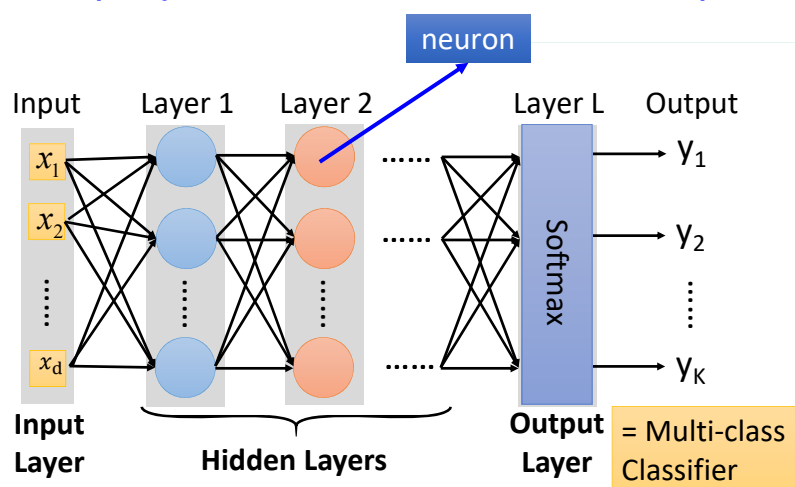
全链接前馈神经网络(Fully Connect Feedforward Network):



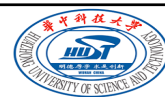
10.2 神经网络模型参数空间



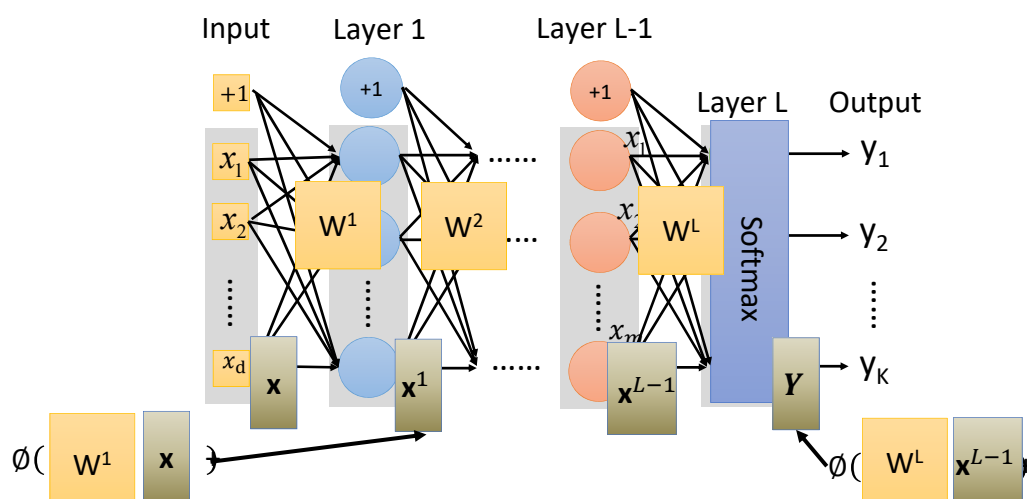
全链接前馈神经网络(Fully Connect Feedforward Network):



10.2 神经网络模型参数空间



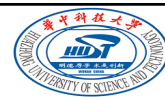
全链接前馈神经网络(Fully Connect Feedforward Network):



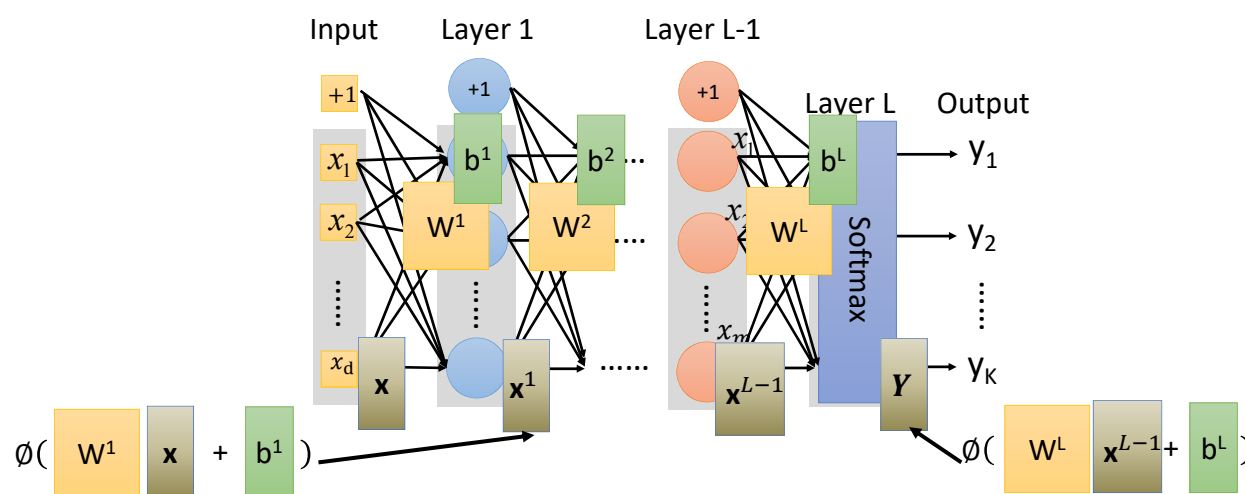
人工智能与自动化学院

19

10.2 神经网络模型参数空间



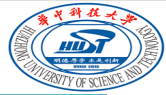
全链接前馈神经网络(Fully Connect Feedforward Network):



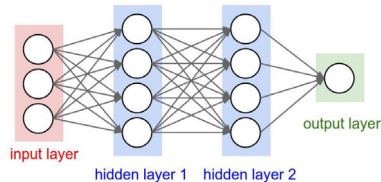
人工智能与自动化学院

20

10.2 神经网络模型参数空间



神经网络模型:

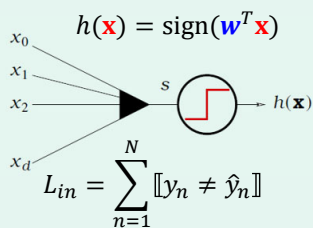


为简单起见，输出端考虑二分类问题
为简单起见，输出层选择线性回归模型

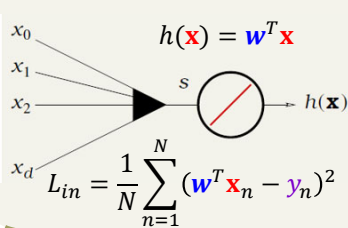
$$\text{Output: } S = \mathbf{w}^T \phi^{(2)}(\phi^{(1)}(\mathbf{x}))$$

$$L_{in} = \frac{1}{N} \sum_{n=1}^N (S - y_n)^2$$

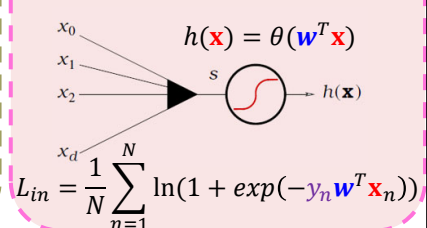
线性分类(感知器):



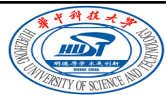
线性回归:



逻辑斯蒂回归:



10.2 神经网络模型参数空间



激活(activation)/变换(transformation)函数:

激活函数可以是任意形式吗?



线性网络，作用不大

$$s = \mathbf{w}^{(3)}(\mathbf{w}^{(2)}(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}) + \mathbf{b}^{(3)}$$

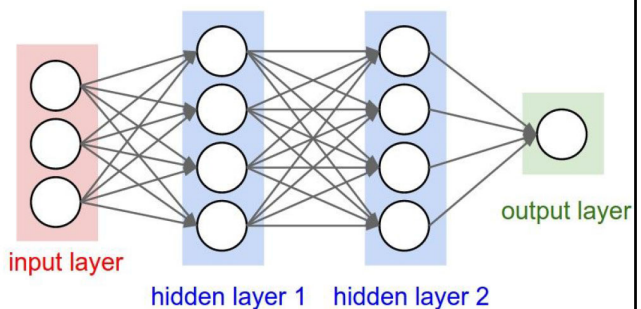
$$= \mathbf{W}\mathbf{x} + \mathbf{b}$$



NP难问题，难以获得最佳的 $\mathbf{w}$

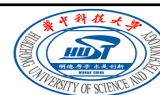


非线性变换，增强网络的性能



$$s = \mathbf{w}^{(3)}\phi(\mathbf{w}^{(2)}\phi(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}) + \mathbf{b}^{(3)}$$

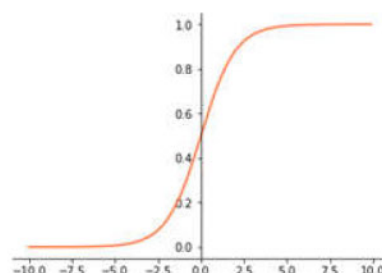
10.2 神经网络模型参数空间



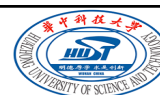
激活(activation)/变换(transformation)函数:

Sigmoid

$$1/(1 + e^{-x})$$



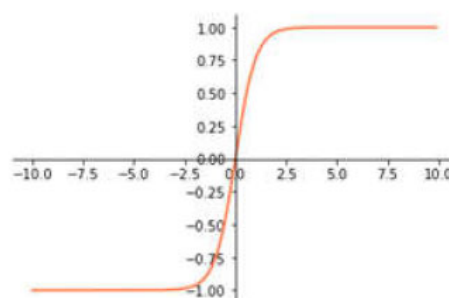
10.2 神经网络模型参数空间



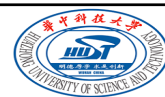
激活(activation)/变换(transformation)函数:

tanh

$$\frac{e^x - e^{-x}}{e^x + e^{-x}}$$



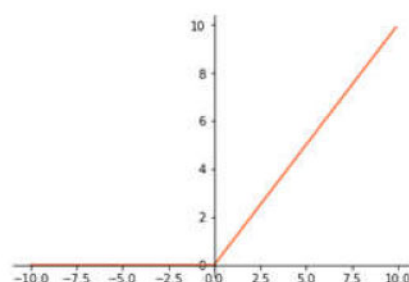
10.2 神经网络模型参数空间



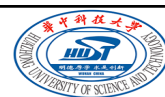
激活(activation)/变换(transformation)函数:

ReLU

$$\max(0, x)$$



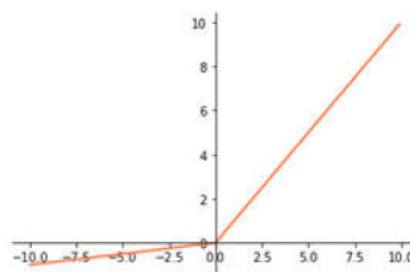
10.2 神经网络模型参数空间



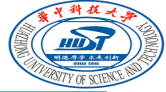
激活(activation)/变换(transformation)函数:

Leaky ReLU

$$\max(0.1x, x)$$



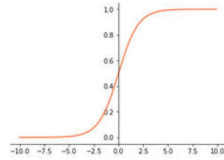
10.2 神经网络模型参数空间



激活(activation)/变换(transformation)函数:

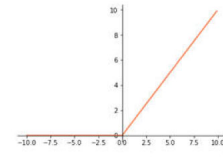
Sigmoid

$$1/(1 + e^{-x})$$



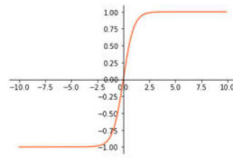
ReLU

$$\max(0, x)$$



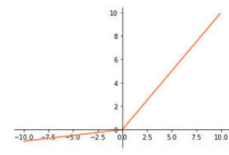
tanh

$$\frac{e^x - e^{-x}}{e^x + e^{-x}}$$



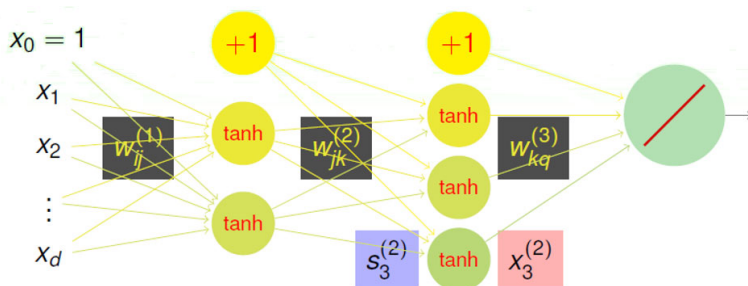
Leaky ReLU

$$\max(0.1x, x)$$



- 每一个感知器单元都应具有非线性特性
- 非线性函数应该具有可导性质便于训练

10.2 神经网络模型参数空间



符号含义:

$d^{(l)}$: 网络的第 l 层

$w_{ij}^{(l)}$: 网络的第 $l-1$ 层到第 l 层的权系数矩阵

$s_j^{(l)}$: 网络第 l 层第 j 个神经元的输入

$x_j^{(l)}$: 网络第 l 层第 j 个神经元的输出

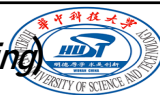
$$w_{ij}^{(l)}: \begin{cases} 1 \leq l \leq L & \text{layers} \\ 0 \leq i \leq d^{(l-1)} & \text{inputs} \\ 1 \leq j \leq d^{(l)} & \text{outputs} \end{cases}$$

$$s_j^{(l)} = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}$$

$$x_j^{(l)} = \begin{cases} \tanh(s_j^{(l)}) & \text{if } l < L \\ s_j^{(l)} & \text{if } l = L \end{cases}$$

将输入样本 $\mathbf{x}$ 作为input layer, 经过hidden layers得到 $\mathbf{x}^{(l)}$, 在output layer去预测 $x_1^{(L)}$

第十讲 神经网络到深度学习 (*From Neural Networks to Deep Learning*)



10.1 神经网络动机 (*Motivation of Neural Networks*)

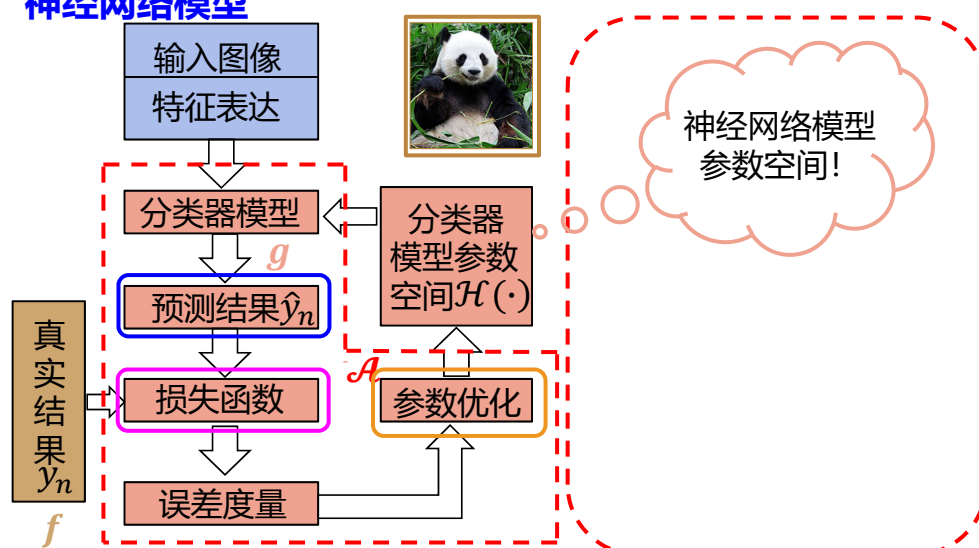
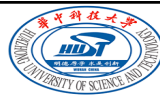
10.2 神经网络模型 (*Neural Network Hypothesis*)

10.3 神经网络学习 (*Neural Network Learning*)

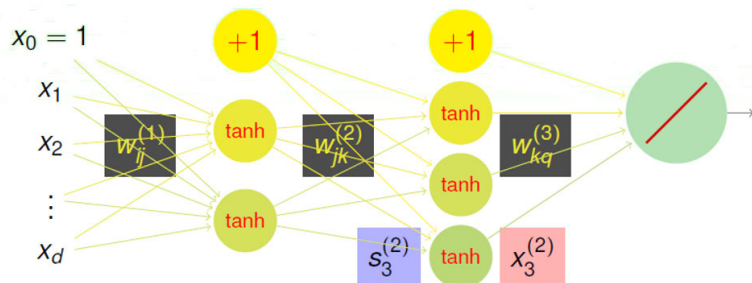
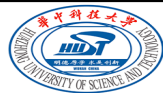
10.4 深度神经网络 (*Deep Neural Networks*)

10.3 神经网络学习

神经网络模型



10.3 神经网络学习



符号含义：

$d^{(l)}$ ：网络的第 l 层

$w_{ij}^{(l)}$ ：网络的第 $l-1$ 层到第 l 层的权系数矩阵

$S_j^{(l)}$ ：网络第 l 层第 j 个神经元的输入

$x_j^{(l)}$ ：网络第 l 层第 j 个神经元的输出

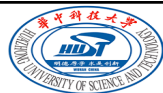
$$w_{ij}^{(l)} : \begin{cases} 1 \leq l \leq L & \text{layers} \\ 0 \leq i \leq d^{(l-1)} & \text{inputs} \\ 1 \leq j \leq d^{(l)} & \text{outputs} \end{cases}$$

$$S_j^{(l)} = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}$$

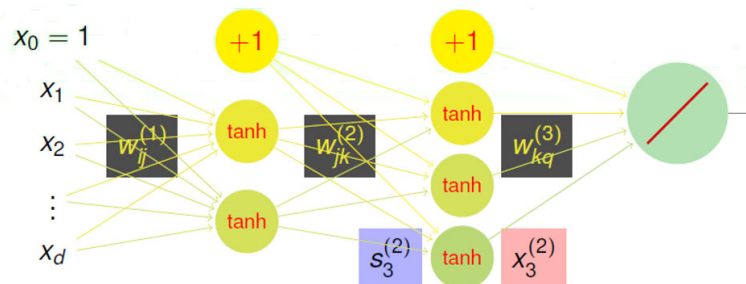
$$x_j^{(l)} = \begin{cases} \tanh(S_j^{(l)}) & \text{if } l < L \\ S_j^{(l)} & \text{if } l = L \end{cases}$$

将输入样本 $\mathbf{x}$ 作为input layer, 经过hidden layers得到 $\mathbf{x}^{(l)}$, 在output layer去预测 $x_1^{(L)}$

10.3 神经网络学习



如何学习网络权重 $w_{ij}^{(l)}$?



$d^{(l)}$ ：网络的第 l 层

$w_{ij}^{(l)}$ ：网络的第 $l-1$ 层到第 l 层的权系数矩阵

$S_j^{(l)}$ ：网络第 l 层第 j 个神经元的输入

$x_j^{(l)}$ ：网络第 l 层第 j 个神经元的输出

$$w_{ij}^{(l)} : \begin{cases} 1 \leq l \leq L & \text{layers} \\ 0 \leq i \leq d^{(l-1)} & \text{inputs} \\ 1 \leq j \leq d^{(l)} & \text{outputs} \end{cases}$$

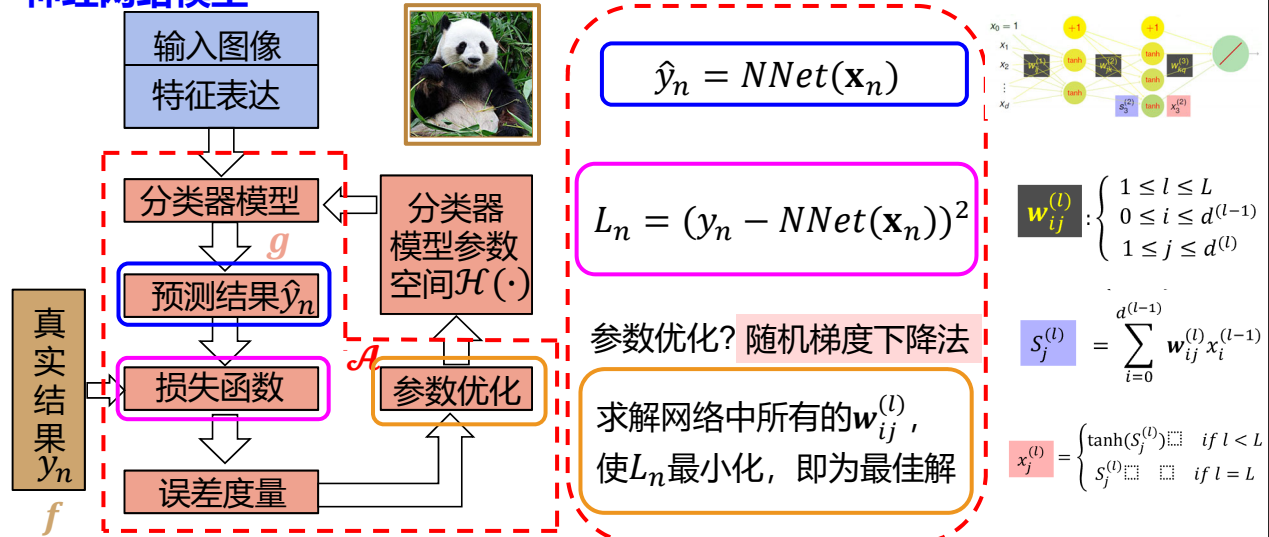
$$S_j^{(l)} = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}$$

$$x_j^{(l)} = \begin{cases} \tanh(S_j^{(l)}) & \text{if } l < L \\ S_j^{(l)} & \text{if } l = L \end{cases}$$

将输入样本 $\mathbf{x}$ 作为input layer, 经过hidden layers得到 $\mathbf{x}^{(l)}$, 在output layer去预测 $x_1^{(L)}$

10.3 神经网络学习

神经网络模型



人工智能与自动化学院

33

10.3 神经网络学习

神经网络中如何求解梯度: $\frac{\partial L_n}{\partial w_{ij}^{(l)}}$

$$L_n = (y_n - NNet(\mathbf{x}_n))^2 = (y_n - S_1^{(L)})^2 = (y_n - \sum_{i=0}^{d^{(L-1)}} w_{i1}^{(L)} x_i^{(L-1)})^2$$

网络输出层: 第 L 层

$$(0 \leq i \leq d^{(L-1)}, j = 1, l = L)$$

$$\frac{\partial L_n}{\partial w_{i1}^{(L)}} = \frac{\partial L_n}{\partial S_1^{(L)}} \cdot \frac{\partial S_1^{(L)}}{\partial w_{i1}^{(L)}}$$

$$= -2(y_n - S_1^{(L)}) \cdot (x_i^{(L-1)})$$

$$\delta_j^{(L)} = -2(y_n - S_1^{(L)})$$

网络其他层: 第 l 层

$$(0 \leq i \leq d^{(l-1)}, 1 \leq j \leq d^{(l)})$$

$$\frac{\partial L_n}{\partial w_{ij}^{(l)}} = \frac{\partial L_n}{\partial S_j^{(l)}} \cdot \frac{\partial S_j^{(l)}}{\partial w_{ij}^{(l)}}$$

$$= \delta_j^{(l)} \cdot (x_i^{(l-1)})$$

$$\delta_j^{(l)} = ?$$

$$w_{ij}^{(l)}: \begin{cases} 1 \leq l \leq L \\ 0 \leq i \leq d^{(l-1)} \\ 1 \leq j \leq d^{(l)} \end{cases}$$

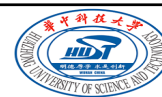
$$S_j^{(l)} = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}$$

$$x_j^{(l)} = \begin{cases} \tanh(S_j^{(l)}) & \text{if } l < L \\ S_j^{(l)} & \text{if } l = L \end{cases}$$

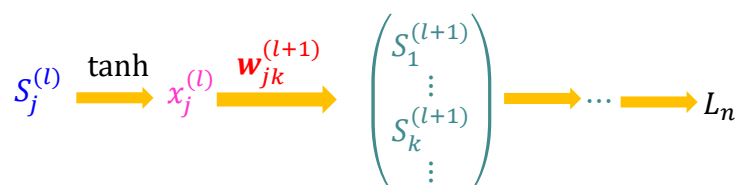
人工智能与自动化学院

34

10.3 神经网络学习



神经网络中如何求解梯度: $\delta_j^{(l)} = \frac{\partial L_n}{\partial S_j^{(l)}}$



$$\delta_j^{(l)} = \frac{\partial L_n}{\partial S_j^{(l)}} = \sum_{k=1}^{d^{(l+1)}} \frac{\partial L_n}{\partial S_k^{(l+1)}} \cdot \frac{\partial S_k^{(l+1)}}{\partial x_j^{(l)}} \cdot \frac{\partial x_j^{(l)}}{\partial S_j^{(l)}}$$

$$= \sum_{k=1}^{d^{(l+1)}} (\delta_k^{(l+1)}) \cdot (w_{jk}^{(l+1)}) \cdot (\tanh(S_j^{(l)}))$$

$\delta_j^{(l)}$ 能够通过反传 $\delta_k^{(l+1)}$ 求得

$$w_{ij}^{(l)} : \begin{cases} 1 \leq l \leq L \\ 0 \leq i \leq d^{(l-1)} \\ 1 \leq j \leq d^{(l)} \end{cases}$$

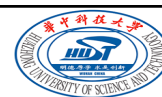
$$S_j^{(l)} = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}$$

$$x_j^{(l)} = \begin{cases} \tanh(S_j^{(l)}) & \text{if } l < L \\ S_j^{(l)} & \text{if } l = L \end{cases}$$

人工智能与自动化学院

35

10.3 神经网络学习



利用反向传播算法实现神经网络学习(Back propagation (Backprop) Algorithm)

初始化所有的权系数 $w_{ij}^{(l)}$

for $t = 0, 1, \dots, T$

- ① 随机性(stochastic): 从训练样本集中任选一个 $\mathbf{x}_n$, $n \in \{1, 2, \dots, N\}$
- ② 前向传播(forward): 利用 $\mathbf{x}^{(0)} = \mathbf{x}_n$ 计算所有的 $x_i^{(l)}$
- ③ 后向传播(backward): 在 $\mathbf{x}^{(0)} = \mathbf{x}_n$ 基础上, 计算所有的 $\delta_j^{(l)}$
- ④ 梯度下降(gradient descent): $w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta x_i^{(l-1)} \delta_j^{(l)}$

返回结果: $g_{NNET}(\mathbf{x}) = \left(\dots \tanh \left(\sum_j w_{jk}^{(2)} \cdot \tanh \left(\sum_i w_{ij}^{(1)} x_i \right) \right) \right)$

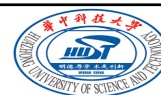
- 所有梯度下降法的技巧都可使用
- 步骤①②③可并行处理, 提高效率

反向传播算法是当前神经网络学习的有效技术

人工智能与自动化学院

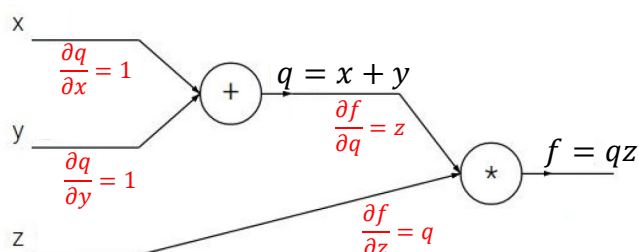
36

10.3 神经网络学习

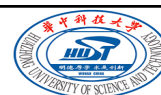


用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):

$$f = (x + y)z$$

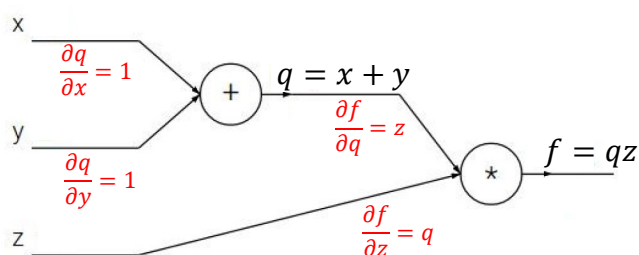


10.3 神经网络学习



用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):

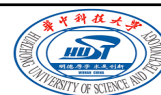
$$f = (x + y)z$$



if: $x = -2, y = 5, z = -4$

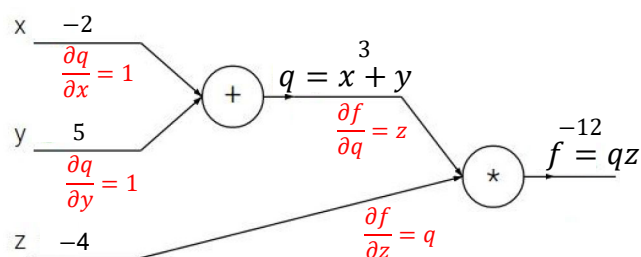
want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

10.3 神经网络学习



用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):

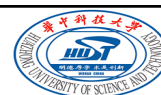
$$f = (x + y)z$$



if: $x = -2, y = 5, z = -4$

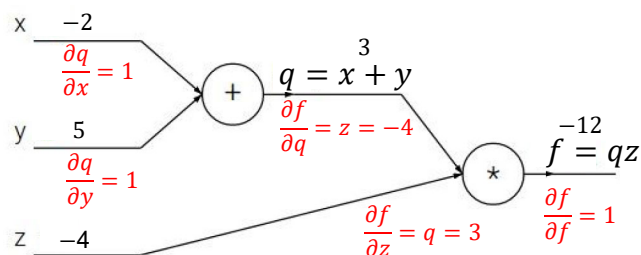
want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

10.3 神经网络学习



用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):

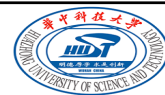
$$f = (x + y)z$$



if: $x = -2, y = 5, z = -4$

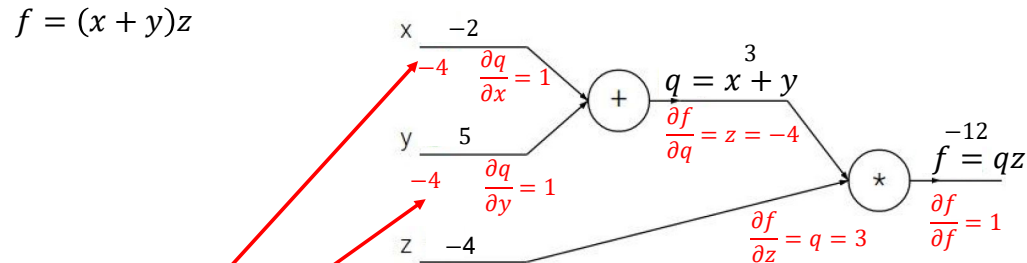
want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

10.3 神经网络学习



用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):

$$f = (x + y)z$$



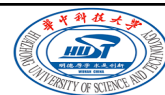
if: $x = -2, y = 5, z = -4$

want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

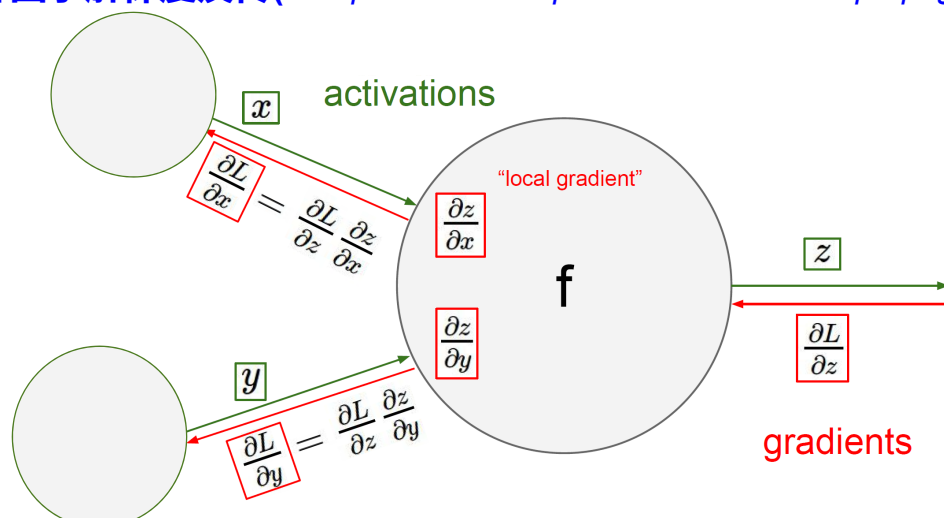
Chain Rule: $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$

$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$

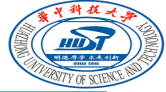
10.3 神经网络学习



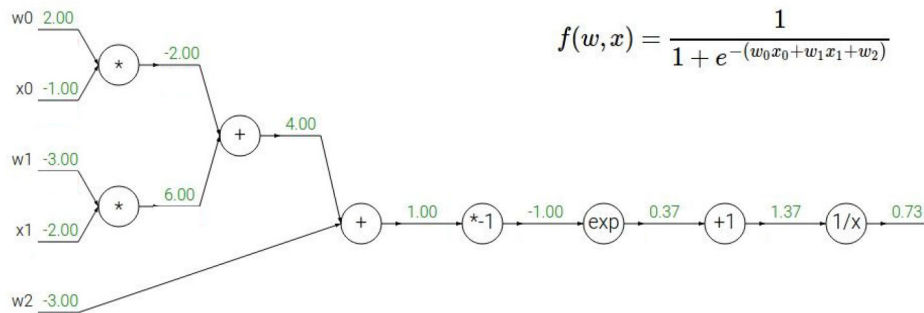
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



10.3 神经网络学习



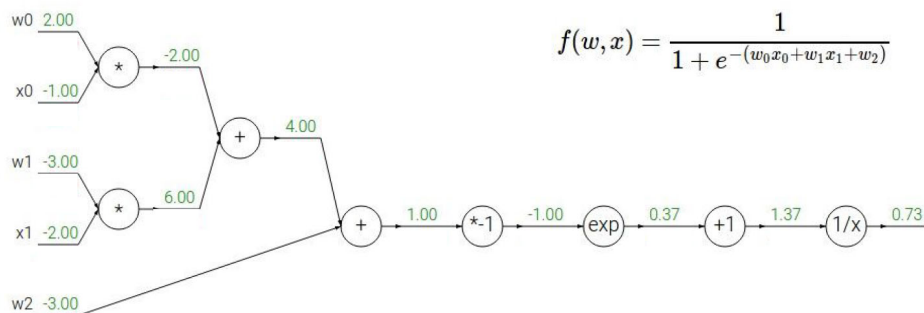
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



10.3 神经网络学习

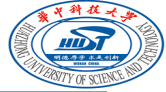


用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):

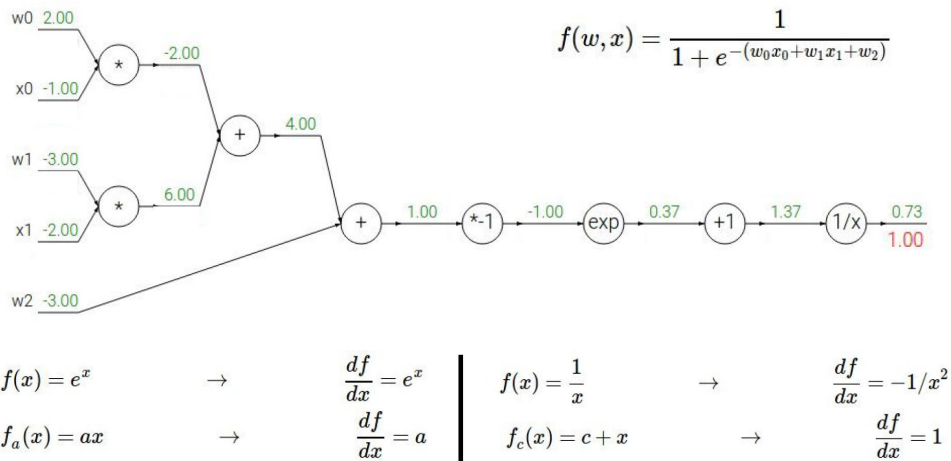


$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$	$\left $	$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$	$\left $	$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

10.3 神经网络学习



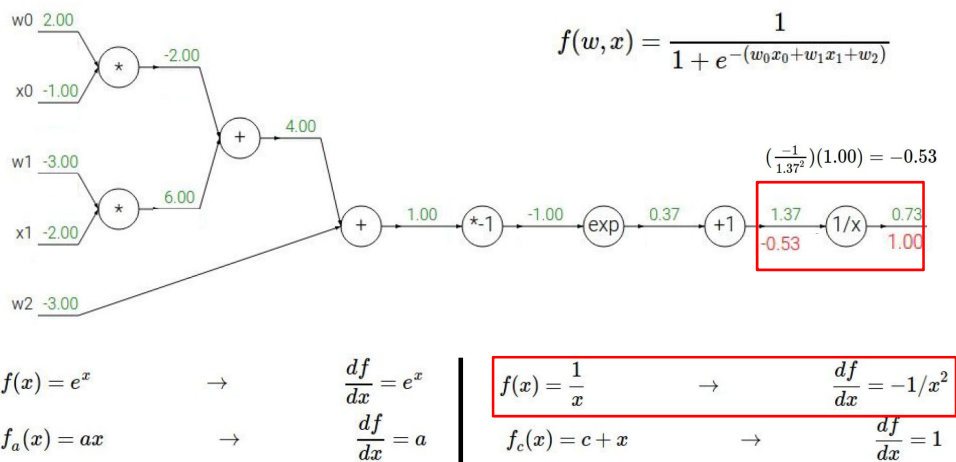
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



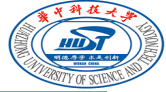
10.3 神经网络学习



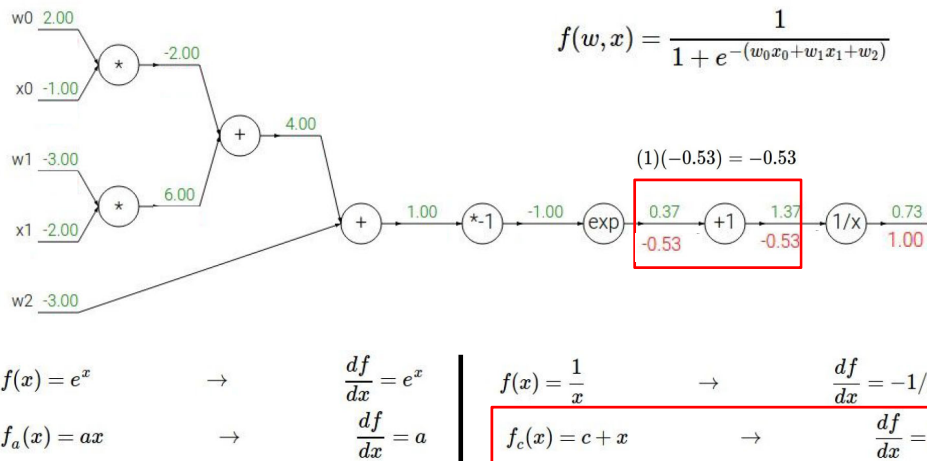
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



10.3 神经网络学习



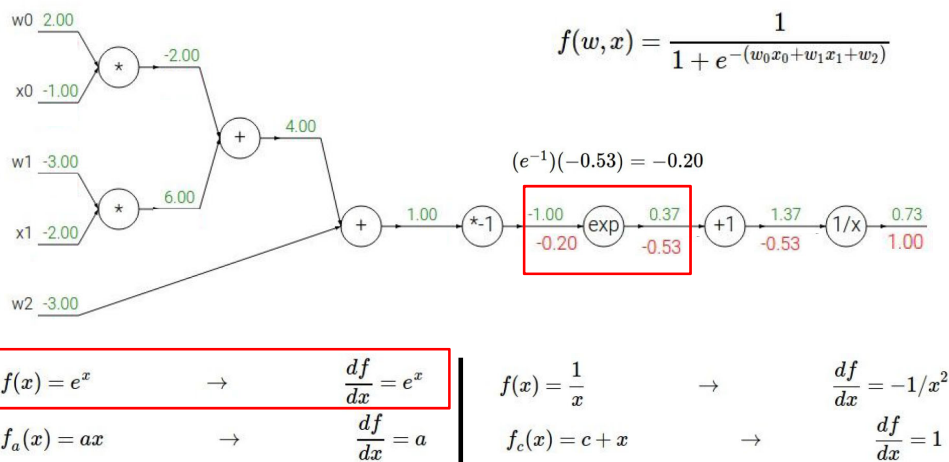
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



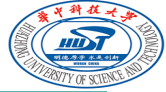
10.3 神经网络学习



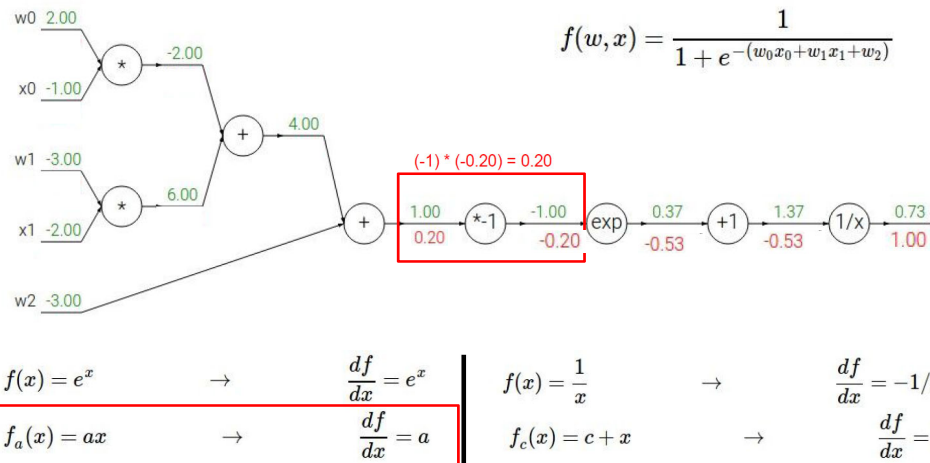
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



10.3 神经网络学习



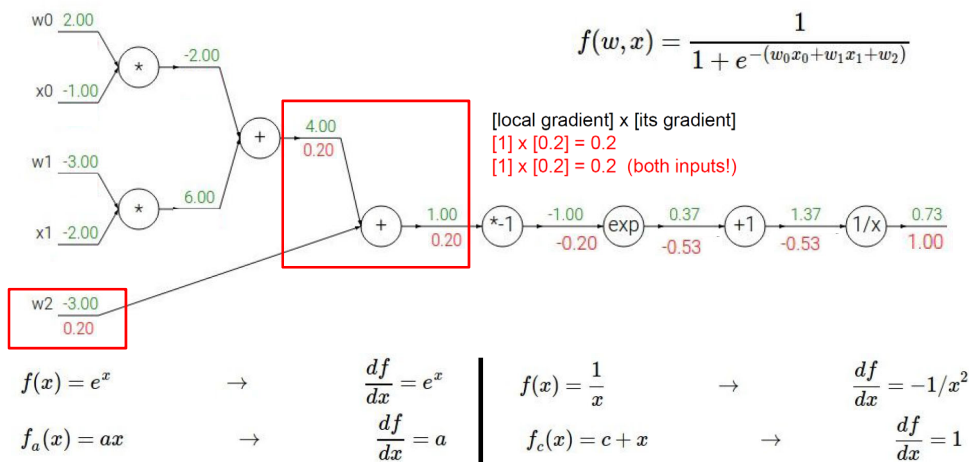
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



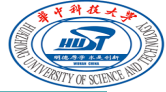
10.3 神经网络学习



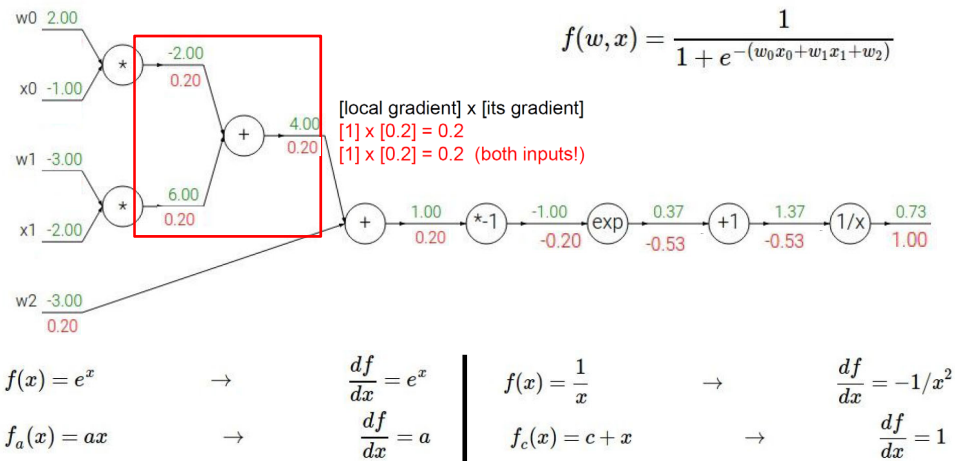
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



10.3 神经网络学习



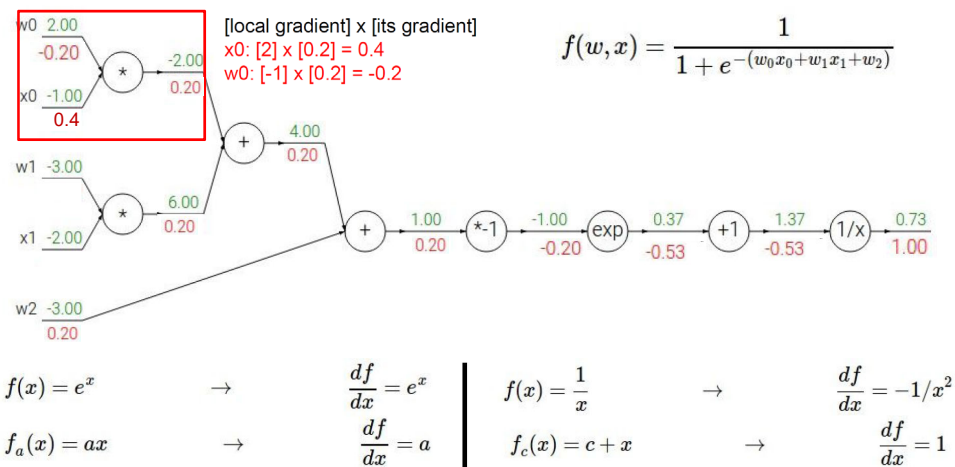
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



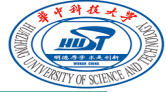
10.3 神经网络学习



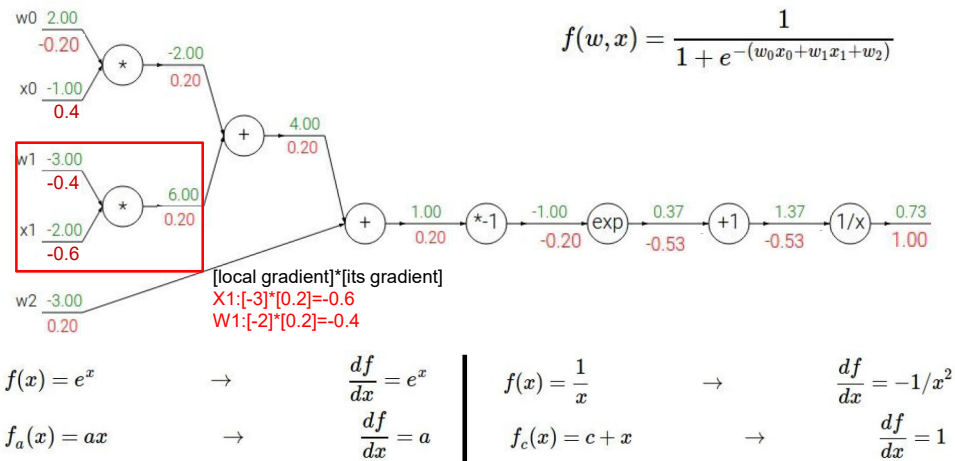
用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



10.3 神经网络学习



用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):



10.3 神经网络学习

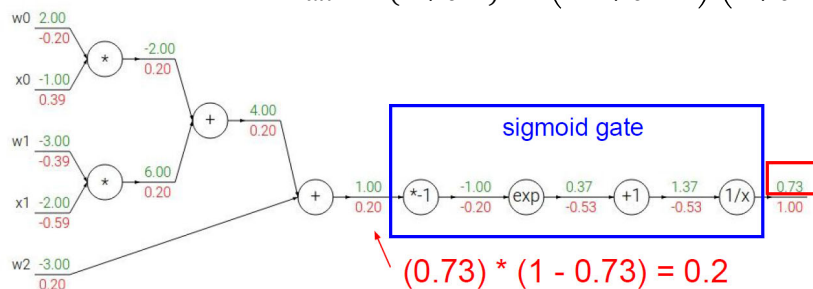


用计算图求解梯度反传(Computational Graph for Gradient Backpropagation):

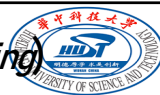
$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

$$\theta(x) = \frac{1}{1 + e^{-x}} \quad \text{sigmoid function}$$

$$\frac{d\theta(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \theta(x))\theta(x)$$



第十讲 神经网络到深度学习(From Neural Networks to Deep Learning)



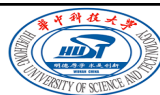
10.1 神经网络动机 (*Motivation of Neural Networks*)

10.2 神经网络模型 (*Neural Network Hypothesis*)

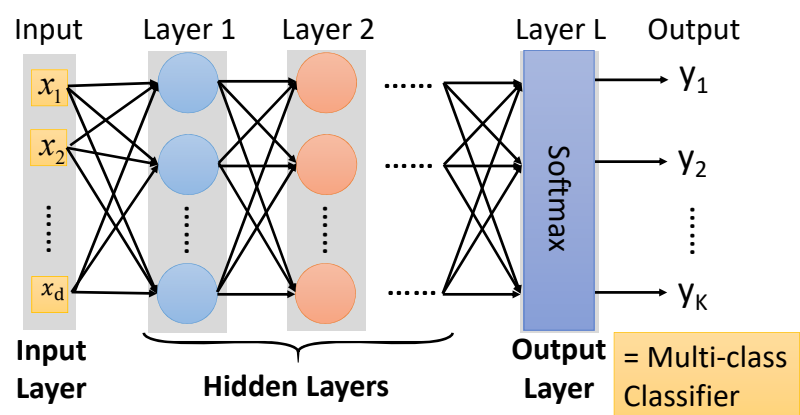
10.3 神经网络学习 (*Neural Network Learning*)

10.4 深度神经网络 (*Deep Neural Networks*)

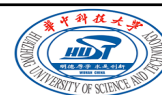
10.4 深度神经网络



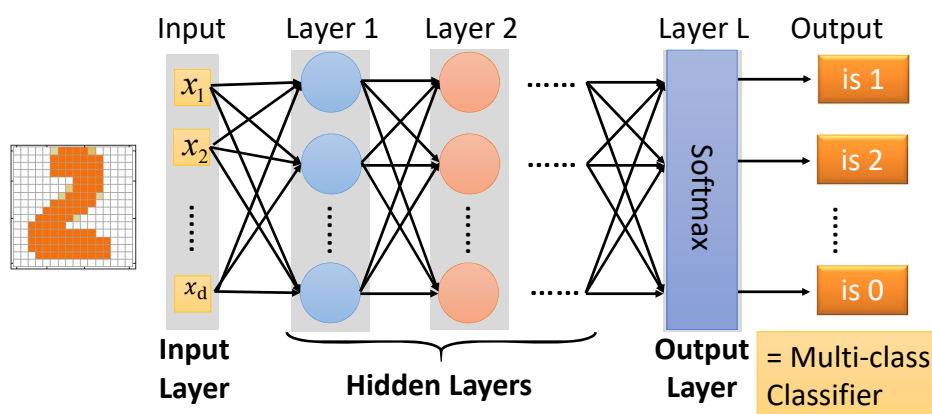
神经网络(*Neural Networks*):



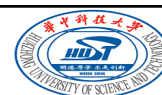
10.4 深度神经网络



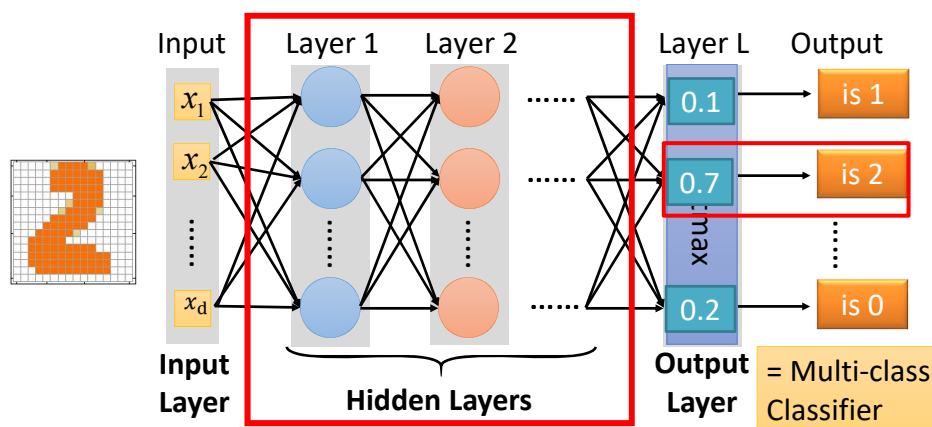
神经网络(Neural Networks):



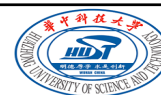
10.4 深度神经网络



神经网络(Neural Networks):

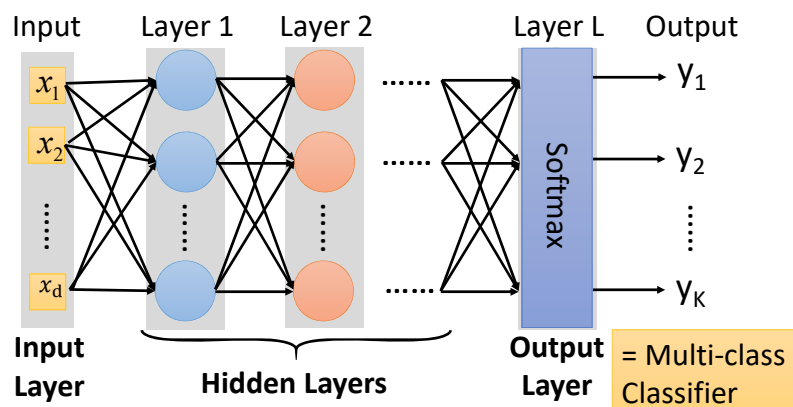


10.4 深度神经网络

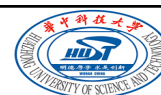


神经网络(Neural Networks):

Deep = Many hidden layers

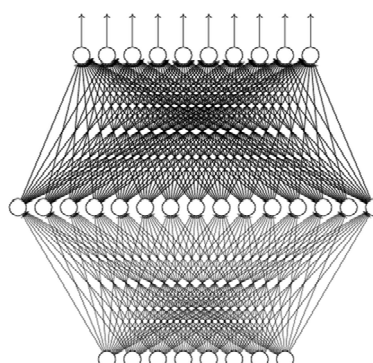


10.4 深度神经网络



为什么要用深度网络?

Layer X Size	Word Error Rate (%)
1 X 2k	24.2
2 X 2k	20.4
3 X 2k	18.4
4 X 2k	17.8
5 X 2k	17.2
7 X 2k	17.1

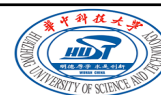


$$f: R^N \rightarrow R^M$$

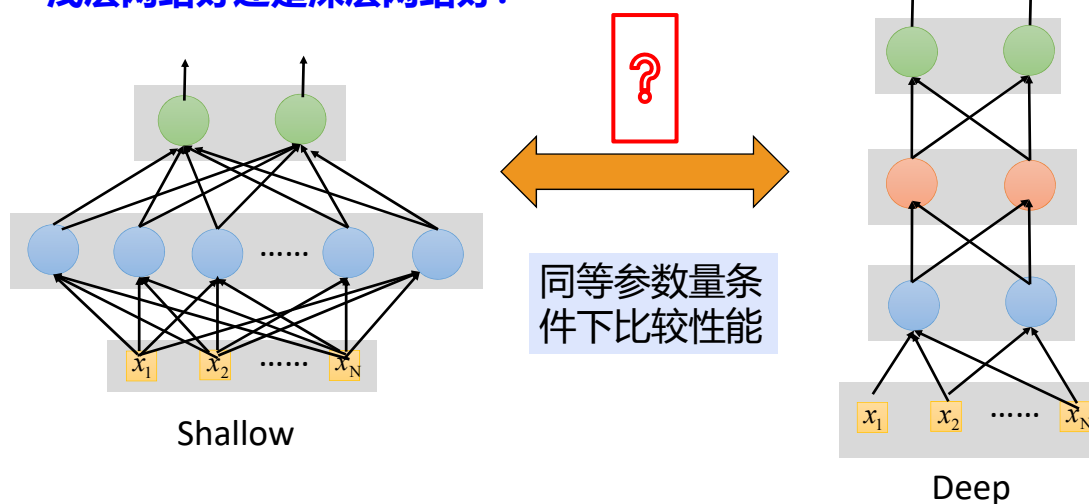
只要有足够多神经元，一个隐含层的网络结构也能拟合出任意复杂的连续曲面

按照学习理论，参数多、性能好

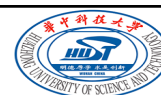
10.4 深度神经网络



浅层网络好还是深层网络好?



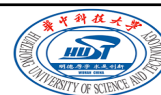
10.4 深度神经网络



为什么要用深度网络?

Layer X Size	Word Error Rate (%)	Layer X Size	Word Error Rate (%)
1 X 2k	24.2		
2 X 2k	20.4		
3 X 2k	18.4		
4 X 2k	17.8		
5 X 2k	17.2	1 X 3772	22.5
7 X 2k	17.1	1 X 4634	22.6
		1 X 16k	22.1

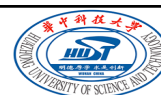
10.4 深度神经网络



Neural Networks practitioner



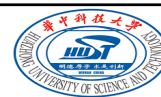
10.4 深度神经网络



深度学习存在的挑战和关键技术

- 如何设计或确定网络结构

10.4 深度神经网络

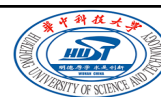


深度学习存在的挑战和关键技术

- 如何设计或确定网络结构



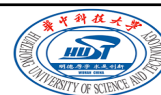
10.4 深度神经网络



深度学习存在的挑战和关键技术

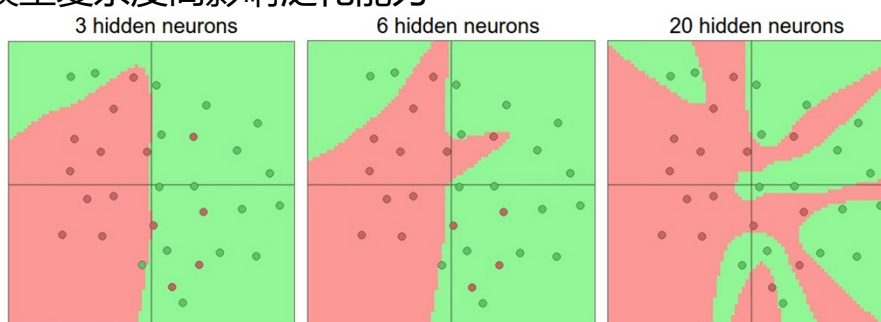
- 如何设计或确定网络结构
 - “领域知识”有助于选择网络结构，如计算机视觉中常常用卷积神经网络

10.4 深度神经网络

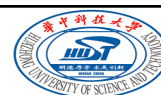


深度学习存在的挑战和关键技术

- 如何设计或确定网络结构
 - “领域知识”有助于选择网络结构，如计算机视觉中常常用卷积神经网络
- 模型复杂度高影响泛化能力



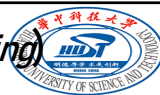
10.4 深度神经网络



深度学习存在的挑战和关键技术

- 如何设计或确定网络结构
 - “领域知识”有助于选择网络结构，如计算机视觉中常常用卷积神经网络
- 模型复杂度高导致过拟合
 - “大数据”有助于增加训练集，提升泛化能力
 - “正则化”——如Dropout等手段，增强对噪声的容忍度
- 模型复杂难以找到最优解
 - 仔细选择初始值，避免落入局部极值，包括预训练等
 - 防止梯度消失，采用合适的激活函数、设计更好的网络结构
- 计算复杂度高
 - 平行化、批量化、GPU等

第十讲 神经网络到深度学习(*From Neural Networks to Deep Learning*)



10.1 神经网络动机 (Motivation of Neural Networks)

多层网络结构具有更强的分类能力

10.2 神经网络模型参数空间 (Neural Network Hypothesis)

基于线性模型构建多层结构

10.3 神经网络学习 (Neural Network Learning)

通过反向传播法有效实现梯度计算

10.4 深度神经网络 (Deep Neural Networks)

介绍了其优越性和面临的挑战